Feasibility Study and Implementation Plan

Travel Itinerary and Expense Organizer (MySQL Workbench implementation)

Prepared by: Team (add names) | Date: 27 October 2026 | Database: `travel_itinerary_db` (version 3, final)

1. Project summary

The Travel Itinerary and Expense Organizer lets a group of travellers plan a trip day by day, book hotels, flights, trains, cabs, buses and metro rides, split costs fairly, and watch the budget. The relational design follows the approved ER diagram and has been extended with a transport module and a smart layer (audit trail and budget alerts). Everything runs in MySQL and is demonstrated in MySQL Workbench.

1.1 The final database at a glance

Item	Count
Tables	32
Foreign keys (each with an explicit ON DELETE and ON UPDATE rule)	41
Triggers	31, grouped into 18 families
Views	4
Stored procedures (transactions)	2
CHECK constraints	26
UNIQUE keys	19
Sample rows	399 across 30 tables (audit_log and trip_alerts start empty and fill when triggers fire)
Named demo queries	25, plus 30 rule and trigger tests
SQL scripts	6 (3 build scripts and 3 demo scripts)

1.2 Three modules, one owner each

Module	Script	Tables	Triggers	Owner
Core and hotel booking: users, destinations, places, trips, itinerary, bookings, expenses, reviews, hotels, amenities, rooms, reservations, payments	`01_travel_itinerary_v2_full.sql`	21	12 (BEFORE)	Member A

Module	Script	Tables	Triggers	Owner
Transport: flights, trains, cabs, buses and metro (public transport)	`03_transport_module.sql`	9	12 (BEFORE)	Member B
Smart layer: audit log, budget alerts, timeline and cost dashboards	`05_smart_features.sql`	2	7 (AFTER)	Member C

1.3 Course concepts the project demonstrates

Concept	Where it appears
ER to relational mapping	32 tables, 41 foreign keys
Strong and weak entities	`hotel_payments`, `hotel_reviews`, `transport_passengers` (identified through their parent)
IS-A specialisation	`places` to `hotels`; `transport_bookings` to five subtype tables
Many-to-many with its own attributes	`hotel_amenities` (is_complimentary), `trip_members` (role), `reviews`
Composite primary and foreign keys	`rooms` references `room_types (room_type_id, hotel_id)`
Referential actions	CASCADE, RESTRICT and SET NULL, each chosen on purpose
Integrity constraints	PRIMARY KEY, UNIQUE, CHECK, NOT NULL, DEFAULT, ENUM
Triggers	24 BEFORE triggers that reject bad data, 7 AFTER triggers that react
Views and derived data	`v_reservation_summary`, `v_trip_transport`, `v_trip_timeline`, `v_trip_cost_summary`

Concept	Where it appears
Transactions and locking	`sp_create_trip`, `sp_book_room` with `SELECT ... FOR UPDATE`
Query techniques	Joins, outer joins, anti-joins, subqueries, GROUP BY / HAVING, UNION ALL, window functions, GROUP_CONCAT

1.4 Objectives

1. Implement every entity of the ER diagram with the same keys, constraints and referential actions.
2. Extend the design with real transport (flights, trains, cabs, public buses and metro) and with an audit and alert layer.
3. Load realistic sample data that exercises every relationship, including edge cases.
4. Prove that the design works: queries return correct answers and the database itself rejects invalid data.
5. Document the plan, technology, team split and every trigger so that each member can explain the work to the examiner.

2. Feasibility analysis

2.1 Technical feasibility: feasible

- Every feature of the design has a native MySQL equivalent: composite keys, ENUM, exact DECIMAL money types, CHECK constraints (enforced from MySQL 8.0.16), BEFORE and AFTER triggers with SIGNAL, views, transactions and row locking.
- Derived values (nights, total cost, balance due, journey duration, budget used) are never stored; they are computed in views and queries, which keeps the schema normalised.
- Rules that a CHECK constraint cannot express because they need other rows or tables (overlapping activities, double-booked rooms, carrier type versus mode) are implemented as triggers.
- Risk level: low. All techniques belong to a standard DBMS course.

2.2 Operational feasibility: feasible

A team of three needs only a laptop each with MySQL Server and MySQL Workbench. Setup takes under 30 minutes. All scripts rebuild the database from scratch, so any member can restore a clean state before the demo.

2.3 Economic feasibility: feasible (cost: Rs 0)

MySQL Community Edition, MySQL Workbench, Python, Streamlit and GitHub are free. The only optional cost is an AI API key for the trigger explainer in section 3.4; the project works fully without it.

2.4 Schedule feasibility: feasible

The database layer is already scripted. The remaining work is running the scripts, collecting screenshots, drawing the EER diagram, the optional app and rehearsal. The roadmap in section 3.6 fits in 14 days.

2.5 Data and legal feasibility: feasible

- All people, hotels, drivers and phone numbers are fictional, and e-mail addresses use the `.example` domain. Carrier and station names are real organisations used only as illustrative labels; every time, number and fare is sample data, not a live schedule or price.
- Passwords in the demo are SHA2 hashes. A real application must use bcrypt or argon2 with a salt; this is stated as a limitation.
- No card numbers are stored, only a transaction reference.

2.6 Risk register

Risk	Likelihood	Impact	Mitigation
Server and Workbench version mismatch	Medium	Medium	Use MySQL 8.4 LTS; connect and run the quick-check query on Day 1
CHECK constraints ignored on very old MySQL	Low	High	Require 8.0.16 or newer; the quick check confirms object counts

Risk	Likelihood	Impact	Mitigation
Trigger creation blocked by privileges or binary logging	Low	Medium	Use the root account locally; if error 1419 appears, run `SET GLOBAL log_bin_trust_function_creators = 1;`
Scripts run in the wrong order	Medium	Medium	Fixed order 01, 03, 05; each script prints its expected counts; re-running 01 means re-running 03 and 05
Workbench stops a script at the first error	Medium	Low	Rule tests are run one statement at a time inside a transaction that is rolled back
A teammate edits the schema separately	Medium	High	One GitHub repository; each module has one owner; changes go through pull requests
One member is unable to present	Low	High	Cross-review: every member explains one other module (section 3.7)

Verdict: the project is technically, operationally, economically and schedule-wise feasible. Recommendation: proceed.

3. Technology stack and roadmap

3.1 Core stack (required)

Layer	Choice	Why
Database server	MySQL Community Server 8.4 LTS	MySQL 8.0 reached end of life in April 2026, so 8.4 LTS is the supported long-term release. The scripts also run on 8.0.16 or newer.

Layer	Choice	Why
Client and modelling	MySQL Workbench	Runs the scripts, shows result grids and draws the EER diagram. The new Workbench 26.x line was announced in September 2026 and its announcement did not list the EER editor, so check your version; if the editor is missing, install classic Workbench 8.0.x only for the diagram.
Language	SQL, MySQL dialect	DDL, DML, DQL, TCL, plus stored program language for triggers and procedures
Storage engine	InnoDB	Needed for foreign keys, transactions and row locking
Character set	utf8mb4	Full Unicode
Version control	Git and GitHub	Shared repository, pull requests, one owner per module

3.2 Why this stack and not another

Alternative	Why it was not chosen
PostgreSQL	Excellent, but the course tooling is MySQL Workbench; switching adds risk without adding marks
SQLite	No stored procedures and no SIGNAL, so the business rules could not live in the database
MongoDB	The data is strongly relational and relies on constraints and joins; a document store would drop the main learning goals
Firebase or Supabase	Hides the schema work that is being graded

3.3 Optional application layer (Phase 4)

A small web front end is not required for a Workbench submission but makes the demo much stronger. Business rules stay inside the database, so the app is thin.

Package	Purpose
Python 3.12	Application language
`streamlit`	Fast web UI without front-end code: dashboard, timeline, booking forms
`mysql-connector-python`	Official driver; plain SQL keeps the queries visible to the examiner
`pandas`	Query results as tables
`plotly`	Budget-versus-spend and mode-share charts
`python-dotenv`	Keeps credentials out of the code
`anthropic` (optional)	Calls the Claude API for the trigger explainer
Docker Compose (optional)	Starts a MySQL 8.4 container that loads scripts 01, 03 and 05 automatically, so the demo works on any laptop

Suggested pages: **Dashboard** (`v_trip_cost_summary` with alerts), **Timeline** (`v_trip_timeline`), **Hotels** (free-room search, `sp_book_room`), **Transport** (`v_trip_transport`), **Expenses** (settlement query), **Rule Lab** (try to break a rule and show the database error) and **Trigger Explainer** (section 3.4).

3.4 Optional: AI trigger explainer

The app reads each trigger's source from `information_schema.triggers` and sends it to an AI model with a fixed prompt (the prompt is in section 6.7). The reply is shown as a plain-English explanation. Cautions: it needs an API key and small usage cost, the answer must be checked against the real code before it is shown to the examiner, and the demo must still work offline, so keep the written explanations of section 6 as the fallback.

3.5 Repository layout

```
travel-itinerary-db/
  sql/        01_travel_itinerary_v2_full.sql  03_transport_module.sql
05_smart_features.sql
  demo/       02_demo_queries.sql  04_transport_demo.sql
06_demo_amenities_views_triggers.sql
  docs/       report, EER diagram image, screenshots, data dictionary
  app/        streamlit app (optional)
  docker/     docker-compose.yml (optional)
  README.md   run order and expected quick-check counts
```

3.6 Fourteen-day roadmap

If the deadline is shorter, merge Phases 3 and 5 and drop Phase 4.

Phase	Days	Goal	Technology	Done when
0. Setup	1 to 2	Everyone has a working server, Workbench and the repository	MySQL 8.4, Workbench, Git	Each member's quick check shows 21 tables after script 01
1. Core and hotels	2 to 3	Compare the generated tables with the ER diagram column by column	Workbench, SQL	Member A signs off the comparison table
2. Transport and smart layer	3 to 5	Run scripts 03 and 05; read every trigger	SQL	Quick check shows 32 tables, 41 foreign keys, 31 triggers, 4 views
3. Testing and evidence	5 to 8	Run all three demo files, capture screenshots, generate the EER diagram	Workbench	Every expected result in section 7 matches
4. Application (optional)	8 to 12	Streamlit dashboard, booking form, Rule Lab, trigger explainer	Python, Streamlit, Docker	App runs from a clean clone with one command

Phase	Days	Goal	Technology	Done when
5. Report and viva	12 to 14	Final report, slides, rehearsal on a clean laptop	Docs, slides	Full demo runs in 12 minutes without errors

3.7 Team of three: who does what

Member	Module	Build tasks	Presents in the viva	Reviews
Member A	Core and hotels	Own script 01; verify all 21 tables against the ER diagram; EER diagram; GitHub owner; data dictionary for the core tables	ER diagram, IS-A, weak entities, composite key, amenities, hotel triggers, demo 02 Section C	Member B's transport SQL
Member B	Transport and public transport	Own script 03; 9 tables; 12 transport triggers; `v_trip_transport`; transport data dictionary; demo 04	Supertype and subtypes, public transport, transport triggers, demo 04	Member C's smart-layer SQL
Member C	Smart layer, queries and report	Own script 05; audit and alert triggers; two dashboards; queries; report, slides and the optional app; demo 06	Views, AFTER triggers, alerts and audit demo, query results	Member A's core SQL

Team rules: one branch per member and merge by pull request; a 15-minute sync every second day; nobody edits another member's script without telling the owner; every member must be able to explain all 18 trigger families at overview level using section 6.

4. Database design explained

4.1 The 32 tables by module

Module	Tables
People	users
Catalogue	destinations, places, activity_categories, expense_categories, amenities
Trip planning	trips, trip_members, itinerary_days, itinerary_items, bookings (activity tickets)
Money	expenses
Community	reviews, saved_destinations
Hotel module	hotels, hotel_amenities, room_types, rooms, hotel_reservations, hotel_payments, hotel_reviews
Transport module	carriers, stations, transport_bookings, flight_details, train_details, cab_details, bus_details, metro_details, transport_passengers
Smart layer	audit_log, trip_alerts

4.2 Key design ideas

Idea	How it is implemented	Why
IS-A specialisation	`hotels.hotel_id` is both primary key and foreign key to `places`; each transport subtype table uses `transport_id` the same way	Common data lives once in the supertype; only subtype-specific columns sit in the subtype
Weak entities	`hotel_payments` (reservation_id, payment_no), `hotel_reviews` (reservation_id), `transport_passengers` (transport_id, passenger_no)	They cannot exist without their parent, so the parent's key is part of their key

Idea	How it is implemented	Why
Composite foreign key	`rooms (room_type_id, hotel_id)` references `room_types (room_type_id, hotel_id)`	A room can never be given a room type that belongs to another hotel
Price snapshot	`hotel_reservations.rate_per_night`	Later changes to a room type's base rate must not change past bookings
Derived data in views	nights, total cost, balance due, duration, budget used	Never stored, so never inconsistent
Referential actions	CASCADE for owned children (a trip's days, items, legs), RESTRICT for shared master data (a carrier or category in use, a user who owns trips), SET NULL for optional links (an itinerary item or place that is deleted)	Deleting a trip cleans up after itself, but important reference data cannot disappear by accident
Exact money	DECIMAL(10,2)	FLOAT would cause rounding errors
Fixed value lists	ENUM for statuses, roles, modes	Invalid values are rejected by the column itself

4.3 The role of the Amenities table

In the ER diagram `amenities` is a strong lookup entity: the master list of facilities (Wi-Fi, Swimming Pool, Breakfast, Parking, Spa, Gym, Airport Shuttle, Air Conditioning), each stored once with its own key. `hotel_amenities` is the associative entity that resolves the many-to-many relationship between hotels and amenities. Its primary key is (hotel_id, amenity_id), both foreign keys use ON DELETE CASCADE, and it carries one attribute of its own, `is_complimentary`.

Why it is built this way

- A hotel has many amenities and an amenity belongs to many hotels, so a direct foreign key in either table would be wrong. A junction table is the standard solution.
- `is_complimentary` depends on the pair, not on either side: Wi-Fi can be free at one hotel and paid at another. That is why the attribute sits on the link and not in `amenities` or `hotels`.

- Storing names once avoids typing mistakes and lets queries filter reliably (for example: hotels with free Wi-Fi and free parking).
- Participation is optional on both sides. In the sample data, Gym is offered by no hotel, which shows that an amenity may exist without any link.

```
hotels 1 --- N hotel_amenities N --- 1 amenities
               (hotel_id, amenity_id, is_complimentary)
```

If your reference PDF describes amenities differently. The PDF was not attached to this conversation, so it could not be compared. These are the usual variants; the implemented design is the first one.

Variant	Meaning	What it would change
1. Junction table hotels to amenities (ER diagram, implemented)	Amenities belong to hotels, with a free-or-paid flag	Nothing
2. Amenities attached to room types	Room-level facilities such as air conditioning or balcony	Add a junction table `room_type_amenities (room_type_id, amenity_id)`; the amenities table stays the same
3. Amenities as a column in hotels (list or flags)	Facilities stored as text such as 'Wi-Fi, Pool'	Breaks first normal form (repeating values in one column); the junction table above is the corrected form, so explain it to the examiner as a deliberate refinement

In the viva, state clearly that the submitted ER diagram is the final design and that the implementation follows it. Demo 06 Section A shows four amenity queries and the CASCADE behaviour.

4.4 Where each rule is enforced

Mechanism	Examples
PRIMARY KEY and UNIQUE	unique e-mail; one day per date in a trip; no seat sold twice on a booking; unique confirmation codes; one alert of each type per trip
FOREIGN KEY	41 links with CASCADE, RESTRICT or SET NULL
CHECK	budget at least 0; end date not before start date; rating from 1 to 5; check-out after check-in; arrival after departure; fares and amounts not negative
BEFORE triggers (24)	reject rows that break rules needing other rows or tables (section 6)
AFTER triggers (7)	write the audit log and budget alerts (section 6)
Transactions	`sp_create_trip`: trip, owner membership and daily rows together; `sp_book_room`: lock the room, insert, set the confirmation code
Integrity query	Q12 confirms every trip owner is also a member with role owner

4.5 Normalisation

All tables are in third normal form: every non-key attribute depends on the key, the whole key and nothing but the key. Lookup values live in their own tables, many-to-many links use junction tables, repeating groups (passengers, payments, amenities) are separate rows, and computed values are never stored.

5. Sample data

5.1 Row counts

Table	Rows	Table	Rows
users	6	hotel_amenities	34
destinations	7	room_types	16

Table	Rows	Table	Rows
places	24	rooms	32
activity_categories	7	hotel_reservations	8
expense_categories	6	hotel_payments	10
amenities	8	hotel_reviews	2
trips	5	reviews	8
trip_members	15	saved_destinations	7
itinerary_days	24	hotels	8
itinerary_items	47	bookings	11
expenses	27	**Core subtotal**	**312**
carriers	10	cab_details	5
stations	16	bus_details	3
transport_bookings	14	metro_details	2
flight_details	3	transport_passengers	33
train_details	1	**Transport subtotal**	**87**

Grand total 399 rows. `audit_log` and `trip_alerts` start empty and fill when the AFTER triggers fire.

5.2 The five sample trips

Trip	Dates	Status	Budget (Rs)	What it demonstrates
Goa Beach Escape	26 to 30 Dec 2026	planned	60,000	Two flights, two airport cabs and a city bus; a confirmed hotel, a cancelled reservation, a failed payment
Rajasthan Heritage Tour	10 to 14 Nov 2026	planned	85,000	Train, Jaipur metro, intercity cab and a pending flight; two hotels in two cities

Trip	Dates	Status	Budget (Rs)	What it demonstrates
Munnar Monsoon Getaway	14 to 17 Aug 2026	completed	40,000	Finished trip: settlement query, cab legs, hotel review
Hampi and Coorg Road Trip	19 to 23 Oct 2026	ongoing	35,000	Self-drive trip with no transport bookings; completed and checked-in stays; budget alert demo
Manali Winter Break	8 to 12 Jan 2027	planned	50,000	Delhi metro and Volvo buses both ways (one pending); cascade-delete demo

Deliberate edge cases: a room under maintenance, a skipped itinerary item, items with no place, expenses with and without an itinerary link, payments by card, UPI, net banking and cash, a refunded payment, bus and cab legs without seat numbers, an amenity (Gym) with no hotel.

6. Business rules and triggers explained

This section is written so that each member can explain every trigger to the examiner.

6.1 What a trigger is

A trigger is a small program stored inside the database. MySQL runs it automatically when a row is inserted, updated or deleted, so the rule applies no matter which tool or user changes the data. Think of a door guard: a BEFORE trigger checks your bag before you enter and may refuse you; an AFTER trigger is the receptionist who writes your name in a register once you are inside.

6.2 Vocabulary

Term	Meaning
BEFORE trigger	Runs before the row is saved; it can reject the change
AFTER trigger	Runs after the row is saved; it reacts, for example by logging

Term	Meaning
`NEW` and `OLD`	The row being written (NEW) and the row as it was before (OLD, for UPDATE and DELETE)
`SIGNAL SQLSTATE '45000'`	Raises a user-defined error with our own message; the statement is cancelled
`FOR EACH ROW`	The trigger runs once for every affected row
`DELIMITER $$`	Tells Workbench that a semicolon inside the body does not end the statement
`INSERT IGNORE`	Skips a row that would break a UNIQUE key instead of raising an error; used so an alert is created only once

6.3 Why triggers, when CHECK and foreign keys exist?

Tool	What it can verify	Example it cannot cover
CHECK	Conditions on columns of the same row	Whether a room is already booked (needs other rows)
FOREIGN KEY	That a referenced row exists	Whether the referenced row has the right type or status
Trigger	Anything the database can query, including other rows and tables	None of the rules in this project

6.4 All 31 triggers in 18 families

BEFORE families come as an insert and an update trigger with the same body (suffix `_bi`, `_bu`); the audit and alert families are AFTER triggers.

No.	Trigger family	Table and event	Rule in plain English	Demo test
1	`trg_exp_item_trip`	expenses, BEFORE insert and update	An expense linked to an itinerary item must belong to the same trip	02, C9
2	`trg_hotels_place_type`	hotels, BEFORE	A hotel row must point at a place whose type is Hotel	02, C10
3	`trg_res_validate`	hotel_reservations, BEFORE	Room not in maintenance; guests fit the room type; stay inside the trip dates; booker is owner or editor; no overlapping booking of the same room (cancelled ones ignored)	02, C3 to C7, D3
4	`trg_hreview_completed`	hotel_reviews, BEFORE	Only a completed stay can be reviewed	02, C8
5	`trg_days_validate`	itinerary_days, BEFORE	day_date must equal trip start plus day_number minus 1 and not pass the trip end	02, C2
6	`trg_items_no_overlap`	itinerary_items, BEFORE	Timed activities on one day must not overlap	02, C1

No.	Trigger family	Table and event	Rule in plain English	Demo test
7	`trg_transport_validate`	transport_bookings, BEFORE	Carrier type matches the mode; every mode except cab needs two different stations of the right kind; a linked item belongs to the same trip; booker is owner or editor	04, X1 to X4, X6, X7
8 to 12	`trg_flight_mode`, `trg_train_mode`, `trg_cab_mode`, `trg_bus_mode`, `trg_metro_mode`	the five subtype tables, BEFORE	A subtype row may exist only for a booking of the matching mode	04, X8
13	`trg_trips_audit_au`	trips, AFTER update	Log changes of budget_total and status	06, C4
14	`trg_res_audit_au`	hotel_reservations, AFTER update	Log status changes	06, C4
15	`trg_transport_audit_au`	transport_bookings, AFTER update	Log status changes	06, C4
16	`trg_exp_audit_au`	expenses, AFTER update	Log amount changes	06, C4
17	`trg_exp_audit_ad`	expenses, AFTER delete	Log the deleted amount	06, C4

No.	Trigger family	Table and event	Rule in plain English	Demo test
18	`trg_exp_budget_alert_ai` and `_au`	expenses, AFTER insert and update	When spending reaches 80 percent of the budget, or passes it, create one alert of that type per trip	06, C1 to C3

Count check: 12 (core) + 12 (transport) + 7 (smart) = 31 triggers.

6.5 Three triggers in depth

(a) `trg_items_no_overlap`: no clashing activities

```
CREATE TRIGGER trg_items_no_overlap_bi BEFORE INSERT ON itinerary_items FOR
EACH ROW
BEGIN
  DECLARE v_cnt INT;
  IF NEW.start_time IS NOT NULL AND NEW.end_time IS NOT NULL THEN
    SELECT COUNT(*) INTO v_cnt FROM itinerary_items i
     WHERE i.day_id = NEW.day_id
       AND i.item_id <> COALESCE(NEW.item_id, 0)
       AND i.start_time IS NOT NULL AND i.end_time IS NOT NULL
       AND NEW.start_time < i.end_time AND NEW.end_time > i.start_time;
    IF v_cnt > 0 THEN
      SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Timed items on the same day
must not overlap';
    END IF;
  END IF;
END
```

1. It fires before every insert (and, in its twin, every update) on `itinerary_items`.
2. If the new item has no times, nothing is checked.
3. It counts other timed items of the same day whose time range overlaps. Two ranges overlap when the new start is before the other's end and the new end is after the other's start. Because the comparison is strict, back-to-back items (10:00 to 12:00, then 12:00 to 13:00) are allowed.

4. `COALESCE(NEW.item_id, 0)` handles two cases: on INSERT the new id is not assigned yet (0 matches nothing), and on UPDATE the row must not be compared with itself.
5. If the count is above zero, SIGNAL raises error 1644 and the row is not saved.

(b) `trg_res_validate` : a safe hotel booking. Cancelled reservations skip all checks. Otherwise it runs five checks in order: the room is not under maintenance; the guests fit the room type's capacity; the stay lies inside the trip dates (check-out may be the morning after the last day); the booker is an owner or editor of the trip; and no other live reservation of the same room overlaps. The overlap rule is the same formula as above applied to dates, so a guest may check in on the day another guest checks out.

(c) `trg_exp_budget_alert_ai` : a reacting trigger. After a new expense is saved, the trigger reads the trip's budget, adds up all its expenses, and compares. Above the budget it inserts a `budget_exceeded` alert; at 80 percent or more it inserts `budget_80`. The table `trip_alerts` has a UNIQUE key on (trip_id, alert_type) and the trigger uses `INSERT IGNORE`, so the same alert is never created twice. It must be an AFTER trigger because the new expense has to exist already to be counted in the total.

6.6 Questions the examiner may ask about triggers

Question	Answer
Trigger or stored procedure?	A procedure runs only when someone calls it. A trigger runs automatically, so it cannot be forgotten or bypassed.
Why BEFORE for validation and AFTER for logging?	BEFORE can still stop the row. AFTER runs when the row is certain to exist, which is what an audit entry or a total needs.
What does SIGNAL do?	It raises an error with our message; the statement is cancelled and no partial row is saved. Inside a transaction the application decides whether to roll back earlier statements.
Why SQLSTATE 45000?	It is the standard code for an unhandled user-defined exception.
Can a trigger change the table it fires on?	No (MySQL error 1442). That is why the alert trigger writes to another table, `trip_alerts`.

Question	Answer
Do cascaded foreign key deletes fire triggers?	No. InnoDB cascades do not activate triggers, which is why audit triggers cover direct updates and deletes only.
What about performance?	Each trigger does a small indexed lookup per row. Bulk loads are slower, normal use is unaffected.
Why not check in the application?	Rules in application code depend on every client being correct. The database is the single gatekeeper for all clients.
Can two triggers share one event?	Yes since MySQL 5.7. Order is creation order unless FOLLOWS or PRECEDES is used; our two AFTER UPDATE triggers on expenses are independent.
How do I list or read them?	`SHOW TRIGGERS;`, `information_schema.triggers`, and `SHOW CREATE TRIGGER name;` (demo 06, Section D).
How were they tested?	Rule tests run inside a transaction that is rolled back, so each rejection can be shown without changing the data.

6.7 Asking an AI to explain a trigger

Use `SHOW CREATE TRIGGER trg_res_validate_bi;` to copy the source, then give an AI model this prompt:

```
You are a database teacher. Explain this MySQL trigger to a second-year
student
in simple English. Cover: 1) when it fires (table, BEFORE or AFTER,
INSERT/UPDATE/DELETE),
2) what it checks or does, step by step, 3) what error or side effect the
user sees,
4) one example that passes and one that fails, 5) why a CHECK constraint or
foreign
key cannot do the same job. Keep it under 200 words.
Trigger source: <paste the SHOW CREATE TRIGGER output here>
```

Always verify the explanation by running the matching rule test from the demo scripts; an AI answer is a draft, not proof.

7. Queries and expected results

7.1 The 25 named queries

File	Query	What it shows	Techniques
02	Q1	Day-wise itinerary of a trip	4-table join, LEFT JOIN, TIME_FORMAT
02	Q2	Budget versus actual spend per trip	LEFT JOIN, SUM, GROUP BY, COALESCE, NULLIF
02	Q3	Spend by expense category	GROUP BY on two columns
02	Q4	Settlement: who owes whom	Derived tables, CROSS JOIN
02	Q5	Reservation summary	View with nights, total, paid and balance
02	Q6	Free rooms for a date range	NOT EXISTS anti-join, user variables
02	Q7	Hotel performance and ratings	Aggregation over a view
02	Q8	Best-rated destinations	GROUP BY, HAVING, AVG
02	Q9	Running total of spend	Window function SUM() OVER
02	Q10	Trip members and roles	GROUP_CONCAT
02	Q11	Transport or adventure items without a booking	LEFT JOIN with IS NULL
02	Q12	Owner-is-member integrity check	Outer join as an audit
04	T1	All legs of a trip with duration and passengers	View over 9 tables
04	T2	Transport spend per trip and mode	GROUP BY
04	T3	Passenger manifest of a bus	Join across supertype and subtype
04	T4	Transport items with no transport booking	NOT EXISTS

File	Query	What it shows	Techniques
04	T5	Fare per passenger by mode	Aggregation over a view
04	T6	Public transport versus cab versus air	CASE grouping
04	T7	Transport share of the budget	LEFT JOIN
06	A1	How many hotels offer each amenity, free or paid	LEFT JOIN, SUM
06	A2	Hotels with free Wi-Fi and free parking	GROUP BY with HAVING COUNT(DISTINCT)
06	A3	Amenity list per hotel	GROUP_CONCAT
06	A4	Deleting an amenity	ON DELETE CASCADE
06	B1	Chronological timeline of a trip	View built with UNION ALL
06	B2	Budget dashboard	View with derived tables

7.2 Expected results (computed from the sample data)

Q2: budget versus actual

Trip	Budget	Actual spend	Remaining	Used
Goa Beach Escape	60,000	30,000	30,000	50.0%
Rajasthan Heritage Tour	85,000	14,600	70,400	17.2%
Munnar Monsoon Getaway	40,000	30,600	9,400	76.5%
Hampi and Coorg Road Trip	35,000	22,100	12,900	63.1%
Manali Winter Break	50,000	5,500	44,500	11.0%

Q4: settlement for the Munnar trip (total 30,600 shared by three members, 10,200 each)

Member	Paid	Fair share	Net balance
Sneha Iyer	21,900	10,200	+11,700 (gets back)

Member	Paid	Fair share	Net balance
Karthik Reddy	6,000	10,200	-4,200 (owes)
Ananya Das	2,700	10,200	-7,500 (owes)

Q5: reservation summary (view `v_reservation_summary`)

Res.	Status	Nights	Total	Paid	Balance
1 Goa, Seaside Palms	confirmed	4	26,000	18,000	8,000
2 Jaipur	confirmed	2	9,600	9,600	0
3 Udaipur	pending	2	13,000	2,000	11,000
4 Munnar	completed	3	13,500	13,500	0
5 Hampi	completed	2	6,400	6,400	0
6 Coorg	checked_in	2	7,600	4,000	3,600
7 Manali	pending	4	12,000	3,000	9,000
8 Goa, Casa Azul	cancelled	4	19,200	0 (refunded)	0

Q6: free Goa rooms for 26 to 29 December 2026. Six rooms: Seaside Palms 101 and 202; Casa Azul 101, 102, 201 and 202. Room 102 at Seaside Palms is under maintenance and room 201 is reserved, so both are excluded; the cancelled Casa Azul reservation does not block anything.

T5: fare comparison by mode

Mode	Legs	Total fare	Passengers	Fare per passenger
Flight	3	24,000	6	4,000.00
Cab	5	14,200	13	1,092.31
Bus	3	5,040	6	840.00
Train	1	3,000	3	1,000.00
Metro	2	90	5	18.00

T6: public transport (train, bus, metro) is 6 legs costing 8,130 in total, against 3 flights at 24,000 and 5 private cabs at 14,200.

B2: budget dashboard (`v_trip_cost_summary`)

Trip	Hotels booked	Transport booked	Total booked	Booked share of budget	Actual spend
Goa Beach Escape	26,000	19,240	45,240	75.4%	30,000
Rajasthan Heritage Tour	22,600	15,030	37,630	44.3%	14,600
Munnar Monsoon Getaway	13,500	7,000	20,500	51.3%	30,600
Hampi and Coorg Road Trip	14,000	0	14,000	40.0%	22,100
Manali Winter Break	12,000	5,060	17,060	34.1%	5,500

Other expected results: A2 returns 5 hotels (Seaside Palms, Pink City Haveli, Pine Cone Lodge, Misty Hills, Coffee Mist); B1 returns 15 events for the Goa trip (5 transport legs, hotel check-in and check-out, 8 activities); Q12 returns 0 rows. In demo 06, adding 10,000 to the Hampi and Coorg trip raises a `budget_80` alert (32,100 of 35,000) and a further 5,000 raises `budget_exceeded` (37,100); a third expense creates no duplicate.

8. Run order and demo plan

8.1 Run order

Step	File	Purpose	Expected result
1	`01_travel_itinerary_v2_full.sql`	Core schema, triggers, procedures, view, data	Quick check: 21 tables, 27 foreign keys, 12 triggers, 1 view
2	`02_demo_queries.sql`	Core queries and rule tests	Run now, while the counts in its Section A still match step 1 (after steps 3 and 5 they become 32, 41 and 31)

Step	File	Purpose	Expected result
3	`03_transport_module.sql`	Transport module	30 tables, 40 foreign keys, 24 triggers, 2 views
4	`04_transport_demo.sql`	Transport queries and rule tests	See section 7
5	`05_smart_features.sql`	Audit, alerts, dashboards	32 tables, 41 foreign keys, 31 triggers, 4 views
6	`06_demo_amenities_views_triggers.sql`	Amenities, dashboards, AFTER trigger demo	See section 7

Re-running script 01 rebuilds the whole database, so run 03 and 05 again afterwards; re-running 03 requires running 05 again.

8.2 Using Workbench

1. Connect to the local server. File, Open SQL Script, choose the file, press the lightning-bolt button to run the whole script.
2. For demo files, click inside one statement and press Ctrl+Enter (Cmd+Enter on a Mac) to run only that statement.
3. EER diagram: Database, Reverse Engineer, choose `travel_itinerary_db`, then File, Export as PNG or PDF. Workbench draws an IS-A link as a one-to-one relationship, so label the subtype tables on the exported image.
4. Refresh the schema panel after running a build script.

8.3 Twelve-minute demo script

Time	Presenter	Step	Show
0:00	Member A	ER diagram and the three modules	EER image, section 1.2
1:30	Member A	Structure check	Demo 06 quick check: 32, 41, 31, 4

Time	Presenter	Step	Show
2:30	Member A	Hotel rules	Demo 02 C3 (double booking), C4 (capacity); `sp_book_room` D2 and D3
4:30	Member A	Amenities	Demo 06 A1 and A2; explain the junction table
6:00	Member B	Transport model	Demo 04 T1, T5, T6
7:30	Member B	Transport rules	Demo 04 X1 (wrong carrier), X3 (bus between airports), X8 (wrong subtype)
9:00	Member C	Views and dashboards	Demo 06 B1 and B2
10:00	Member C	AFTER triggers	Demo 06 C1 to C4: alert, no duplicate alert, audit rows, rollback
11:30	All	Wrap-up and questions	Section 9

8.4 Troubleshooting

Message	Cause and fix
Error 1419, you do not have the SUPER privilege	Binary logging blocks trigger creation for a non-root user; use root or run `SET GLOBAL log_bin_trust_function_creators = 1;`
Error 1064 near DELIMITER	The script was run in a client that does not understand DELIMITER; run it in Workbench
Error 1215, cannot add foreign key	A script was run out of order; start again from script 01
Error 1046, no database selected	Run `USE travel_itinerary_db;` first
Error 1175, safe update mode	Run `SET SQL_SAFE_UPDATES = 0;` for the session
Check constraint not enforced	MySQL older than 8.0.16; upgrade to 8.4 LTS

9. Questions the examiner may ask

Question	Short answer
How did you convert the ER diagram to tables?	Strong entities became tables, many-to-many relationships became junction tables, weak entities took their parent's key, and subtypes share the supertype's key.
Why is hotels a separate table from places?	A hotel is a kind of place with extra attributes. The shared key means no data is repeated, and a trigger makes sure only places of type Hotel can have a hotel row.
Why is hotel_payments a weak entity?	A payment is meaningless without its reservation, so its key is (reservation_id, payment_no).
What is the composite foreign key for?	It stops a room from being linked to a room type of another hotel.
Why store rate_per_night on the reservation?	It freezes the price at booking time.
Which normal form are the tables in, and why?	Third normal form: no repeating groups, no partial dependencies, no transitive dependencies, and derived values are computed in views.
Why DECIMAL for money?	It is exact; FLOAT can round incorrectly.
How do you stop two people booking the same room?	`sp_book_room` runs in a transaction and locks the room row with `SELECT ... FOR UPDATE`; the validation trigger then rejects any overlapping live reservation.
What do ACID properties mean here?	If any step of `sp_create_trip` fails, the trip, the owner membership and the daily rows are all rolled back.
Why choose CASCADE, RESTRICT or SET NULL?	CASCADE for data owned by a parent, RESTRICT for shared reference data, SET NULL for optional links.

Question	Short answer
What are views for?	They hide complex joins and present derived data such as balance due or journey duration without storing it.
How would the design scale?	Add indexes guided by EXPLAIN, partition large booking tables by date, and move reporting queries to a read replica.
What are the limits of the project?	See section 10.

10. Verification, limitations and future work

10.1 Verification status

- The sample data and every trigger rule were simulated in a separate script before delivery. It confirmed no duplicate keys, no broken foreign keys, no overlapping activities, rooms or seats, correct day dates, capacities and booker roles, matching carrier and station types, and reviews only on completed stays.
- The expected results in section 7 were computed from the same data.
- The SQL was written for MySQL 8.0.16 or newer and 8.4, but it could not be executed on a live MySQL server while it was prepared. The first run on the team's own machine is therefore the acceptance test: the quick-check counts at the end of each build script must match section 8.1. If any statement fails, fix it and re-run before the demo.

10.2 Limitations

- Demo passwords use SHA2; a real system needs bcrypt or argon2.
- The settlement query splits costs equally among all members, including viewers.
- Names of carriers and stations are real, but all times, numbers and fares are sample values, not live data.
- The older generic `bookings` table still holds some records for transport items that are now also stored in `transport_bookings`; new transport must use the transport module.
- Audit triggers record direct updates and deletes only; cascaded deletes do not fire triggers.
- No multi-currency support and no ferry or tram mode.

10.3 Future work

1. Tune indexes from EXPLAIN output on the heaviest queries.
2. Role-based MySQL accounts (read-only analyst, trip editor) with GRANT.
3. A scheduled event that marks trips ongoing or completed from their dates.
4. Ferry and tram modes, and a currency column.
5. The Streamlit application with Rule Lab and AI trigger explainer.

11. Conclusion

The Travel Itinerary and Expense Organizer is feasible with free tools and a team of three. The database covers hotels, flights, trains, cabs, buses and metro; enforces its business rules with 31 triggers, 41 foreign keys and 26 CHECK constraints; keeps an audit trail and raises budget alerts; and is backed by 25 named queries and about 30 rule tests. Each of the three members owns one module and can explain all 18 trigger families. We recommend MySQL 8.4 LTS with MySQL Workbench as the core stack, and treating the Python and Streamlit layer as an optional enhancement.